UNIVERSIDAD NACIONAL DE COLOMBIA

SEDE MEDELLÍN

Programación Orientada a Objetos

Actividad 3: Individual (Valor 10%)

Estudiante:	Kevin Santiago Montoya Chavarria
Docente:	Walter Hugo Arboleda Mazo

Asignatura:	Programación Orientada a Objetos (2026-2S)
Lenguaje de desarrollo:	Python (POO)
Texto guía:	Ejercicios de Programación Orientada a Objetos con Java y UML - Leonardo Bermón Angarita

URL del repositorio en GitHub:

https://github.com/Kevin007ch/POO-Kevin-Santiago-Montoya-Chavarria

Tabla de Contenido

Sección	Descripción	Página
Punto 1	Ejercicio resuelto No. 8.2: Componentes GUI y menú	3
Punto 2	Ejercicio resuelto No. 8.3: Gestión de eventos y validaciones	7
Referencias	Texto guía y repositorio en GitHub	10

INTRODUCCIÓN

El presente informe documenta el desarrollo de la Actividad 3 de la asignatura Programación Orientada a Objetos. Se implementan dos aplicaciones gráficas en Python mediante Tkinter: una calculadora de IMC con menú y cálculo estimado de grasa corporal, y una aplicación para gestionar eventos de teclado y validar entradas. La estructura del informe sigue el formato utilizado en la Actividad 1: enunciado o descripción, código fuente y diagrama UML.

Punto 1

Ejercicio resuelto No. 8.2

Enunciado / descripción: Desarrollar una aplicación gráfica que permita ingresar los datos de un paciente, calcular el Índice de Masa Corporal (IMC), clasificar el resultado y estimar el porcentaje de grasa corporal con la fórmula de Deurenberg. La aplicación incluye un menú superior con opciones para limpiar el formulario, mostrar información y salir.

Código Fuente en Python (POO) — ejercicio_8_2.py

```
import tkinter as tk
from tkinter import ttk, messagebox


class VentanaIMC(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Actividad 3 - Ejercicio 8.2: Cálculo de IMC")
        self.geometry("520x560")
        self.resizable(False, False)

        self.crear_menu()

        main_frame = ttk.Frame(self, padding=20)
        main_frame.pack(fill=tk.BOTH, expand=True)

        ttk.Label(
            main_frame, text="Calculadora de IMC y grasa corporal",
            font=("Arial", 14, "bold")
        ).grid(row=0, column=0, columnspan=2, pady=(0, 15))

        ttk.Label(main_frame, text="Nombre completo:").grid(row=1, column=0, sticky=tk.W, pady=5)
        self.txt_nombre = ttk.Entry(main_frame, width=28)
        self.txt_nombre.grid(row=1, column=1, sticky=tk.E, pady=5)

        ttk.Label(main_frame, text="Edad (años):").grid(row=2, column=0, sticky=tk.W, pady=5)
        self.txt_edad = ttk.Entry(main_frame, width=28)
        self.txt_edad.grid(row=2, column=1, sticky=tk.E, pady=5)

        ttk.Label(main_frame, text="Peso (kg):").grid(row=3, column=0, sticky=tk.W, pady=5)
        self.txt_peso = ttk.Entry(main_frame, width=28)
        self.txt_peso.grid(row=3, column=1, sticky=tk.E, pady=5)

        ttk.Label(main_frame, text="Estatura (m):").grid(row=4, column=0, sticky=tk.W, pady=5)
        self.txt_estatura = ttk.Entry(main_frame, width=28)
        self.txt_estatura.grid(row=4, column=1, sticky=tk.E, pady=5)

        ttk.Label(main_frame, text="Género:").grid(row=5, column=0, sticky=tk.W, pady=5)
        self.cmb_genero = ttk.Combobox(
            main_frame, values=["Hombre", "Mujer"], state="readonly", width=25
        )
        self.cmb_genero.set("Hombre")
        self.cmb_genero.grid(row=5, column=1, sticky=tk.E, pady=5)

        btn_frame = ttk.Frame(main_frame)
        btn_frame.grid(row=6, column=0, columnspan=2, pady=12)
        ttk.Button(btn_frame, text="Calcular", command=self.calcular).pack(side=tk.LEFT, padx=5)
        ttk.Button(btn_frame, text="Limpiar", command=self.limpiar).pack(side=tk.LEFT, padx=5)

        ttk.Label(main_frame, text="Resultados", font=("Arial", 11, "bold")).grid(
            row=7, column=0, columnspan=2, pady=(8, 4)
        )
        self.txt_resultados = tk.Text(main_frame, height=9, width=54, wrap=tk.WORD)
        self.txt_resultados.grid(row=8, column=0, columnspan=2, sticky=tk.EW)

    def crear_menu(self):
        menubar = tk.Menu(self)
        menu_archivo = tk.Menu(menubar, tearoff=0)
        menu_archivo.add_command(label="Limpiar formulario", command=self.limpiar)
        menu_archivo.add_separator()
        menu_archivo.add_command(label="Salir", command=self.destroy)
        menubar.add_cascade(label="Archivo", menu=menu_archivo)

        menu_ayuda = tk.Menu(menubar, tearoff=0)
        menu_ayuda.add_command(label="Acerca de...", command=self.mostrar_acerca_de)
        menubar.add_cascade(label="Ayuda", menu=menu_ayuda)
        self.config(menu=menubar)

    def calcular(self):
        try:
```

```
            nombre = self.txt_nombre.get().strip()
            edad = int(self.txt_edad.get())
            peso = float(self.txt_peso.get())
            estatura = float(self.txt_estatura.get())
            genero = self.cmb_genero.get()

            if edad <= 0 or peso <= 0 or estatura <= 0:
                raise ValueError("La edad, el peso y la estatura deben ser mayores que cero.")

            imc = peso / (estatura ** 2)
            if imc < 16:
                estado = "Delgadez severa"
            elif imc < 17:
                estado = "Delgadez moderada"
            elif imc < 18.5:
                estado = "Delgadez leve"
            elif imc < 25:
                estado = "Peso normal"
            elif imc < 30:
                estado = "Sobrepeso"
            elif imc < 35:
                estado = "Obesidad leve"
            elif imc < 40:
                estado = "Obesidad moderada"
            else:
                estado = "Obesidad mórbida"

            valor_sexo = 1 if genero == "Hombre" else 0
            grasa = (1.20 * imc) + (0.23 * edad) - (10.8 * valor_sexo) - 5.4

            resultado = (
                f"Paciente: {nombre if nombre else 'No especificado'}\\n"
                f"----------------------------------------\\n"
                f"IMC calculado: {imc:.2f} kg/m²\\n"
                f"Diagnóstico IMC: {estado}\\n"
                f"Grasa corporal estimada: {grasa:.2f}%"
            )
            self.txt_resultados.delete("1.0", tk.END)
            self.txt_resultados.insert(tk.END, resultado)
        except ValueError as error:
            messagebox.showerror("Error de entrada", f"Verifique los datos ingresados.\\n{error}")

    def limpiar(self):
        self.txt_nombre.delete(0, tk.END)
        self.txt_edad.delete(0, tk.END)
        self.txt_peso.delete(0, tk.END)
        self.txt_estatura.delete(0, tk.END)
        self.cmb_genero.set("Hombre")
        self.txt_resultados.delete("1.0", tk.END)

    def mostrar_acerca_de(self):
        messagebox.showinfo(
            "Acerca de",
            "Universidad Nacional de Colombia - Sede Medellín\\nProgramación Orientada a Objetos"
        )


if __name__ == "__main__":
    app = VentanaIMC()
    app.mainloop()
```

Punto 1 (continuación)

Diagrama de Clases UML

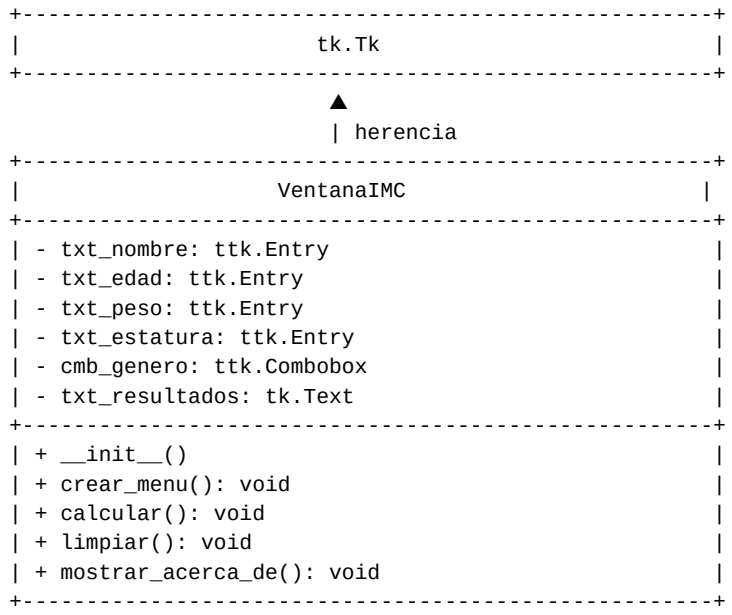
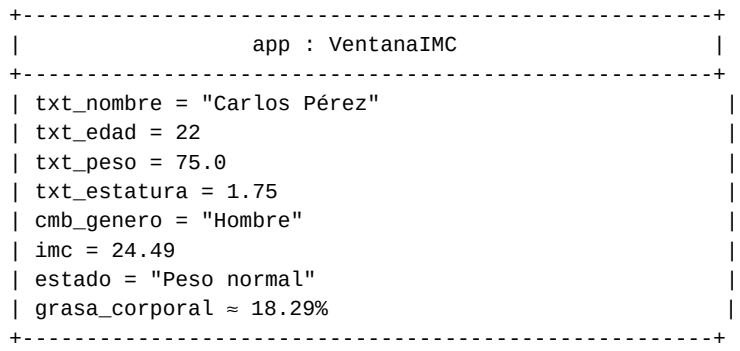


Diagrama de Objetos UML



Ejemplo de salida

Para un paciente de 22 años, 75 kg y 1,75 m de estatura, el IMC es aproximadamente 24,49 kg/m². La clasificación corresponde a peso normal. El porcentaje de grasa se muestra como una estimación calculada por la fórmula incorporada.

Punto 2

Ejercicio resuelto No. 8.3

Enunciado / descripción: Implementar una aplicación que procese eventos de teclado con `bind()`. La validación de peso y estatura se actualiza mientras el usuario escribe mediante el evento `<KeyRelease>`. Se agregan accesos directos: Enter para calcular el IMC y Escape para limpiar los campos.

Código Fuente en Python (POO) — ejercicio_8_3.py

```
import tkinter as tk
from tkinter import ttk


class VentanaEventos(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Actividad 3 - Ejercicio 8.3: Gestión de eventos")
        self.geometry("520x450")
        self.resizable(False, False)

        main_frame = ttk.Frame(self, padding=20)
        main_frame.pack(fill=tk.BOTH, expand=True)

        ttk.Label(
            main_frame, text="Gestión de eventos y validación",
            font=("Arial", 14, "bold")
        ).grid(row=0, column=0, columnspan=2, pady=(0, 8))

        ttk.Label(
            main_frame, text="Atajos: Enter = calcular | Escape = limpiar"
        ).grid(row=1, column=0, columnspan=2, pady=(0, 12))

        ttk.Label(main_frame, text="Peso (kg):").grid(row=2, column=0, sticky=tk.W, pady=5)
        self.txt_peso = ttk.Entry(main_frame, width=28)
        self.txt_peso.grid(row=2, column=1, sticky=tk.E, pady=5)
        self.txt_peso.bind("<KeyRelease>", self.validar_entrada_peso)

        ttk.Label(main_frame, text="Estatura (m):").grid(row=3, column=0, sticky=tk.W, pady=5)
        self.txt_estatura = ttk.Entry(main_frame, width=28)
        self.txt_estatura.grid(row=3, column=1, sticky=tk.E, pady=5)
        self.txt_estatura.bind("<KeyRelease>", self.validar_entrada_estatura)

        self.lbl_status = ttk.Label(main_frame, text="Estado: ingrese los datos.")
        self.lbl_status.grid(row=4, column=0, columnspan=2, pady=8)

        botones = ttk.Frame(main_frame)
        botones.grid(row=5, column=0, columnspan=2, pady=8)
        ttk.Button(botones, text="Calcular IMC", command=self.ejecutar_calculo).pack(side=tk.LEFT, padx=5)
        ttk.Button(botones, text="Limpiar", command=self.limpiar_campos).pack(side=tk.LEFT, padx=5)

        self.txt_salida = tk.Text(main_frame, height=7, width=54, wrap=tk.WORD)
        self.txt_salida.grid(row=6, column=0, columnspan=2, pady=8)

        self.bind("<Return>", lambda event: self.ejecutar_calculo())
        self.bind("<Escape>", lambda event: self.limpiar_campos())

    def es_numero_positivo(self, valor):
        if not valor:
            return True
        try:
            return float(valor) > 0
        except ValueError:
            return False

    def validar_entrada_peso(self, event=None):
        valor = self.txt_peso.get().strip()
        if valor and not self.es_numero_positivo(valor):
            self.lbl_status.config(text="Error: ingrese un peso numérico positivo.")
        else:
            self.lbl_status.config(text="Entrada de peso válida.")

    def validar_entrada_estatura(self, event=None):
        valor = self.txt_estatura.get().strip()
        if valor and not self.es_numero_positivo(valor):
            self.lbl_status.config(text="Error: ingrese una estatura numérica positiva.")
        else:
            self.lbl_status.config(text="Entrada de estatura válida.")

    def ejecutar_calculo(self):
```

```
        try:
            peso = float(self.txt_peso.get())
            estatura = float(self.txt_estatura.get())
            if peso <= 0 or estatura <= 0:
                raise ValueError

            imc = peso / (estatura ** 2)
            salida = (
                "=== CÁLCULO DE IMC ===\\n"
                f"Peso: {peso:.2f} kg\\n"
                f"Estatura: {estatura:.2f} m\\n"
                f"IMC: {imc:.2f}"
            )
            self.txt_salida.delete("1.0", tk.END)
            self.txt_salida.insert(tk.END, salida)
            self.lbl_status.config(text="Cálculo realizado correctamente.")
        except ValueError:
            self.lbl_status.config(text="Error: ingrese peso y estatura válidos y mayores que cero.")

    def limpiar_campos(self):
        self.txt_peso.delete(0, tk.END)
        self.txt_estatura.delete(0, tk.END)
        self.txt_salida.delete("1.0", tk.END)
        self.lbl_status.config(text="Estado: campos limpiados.")


if __name__ == "__main__":
    app = VentanaEventos()
    app.mainloop()
```

Punto 2 (continuación)

Diagrama de Clases UML

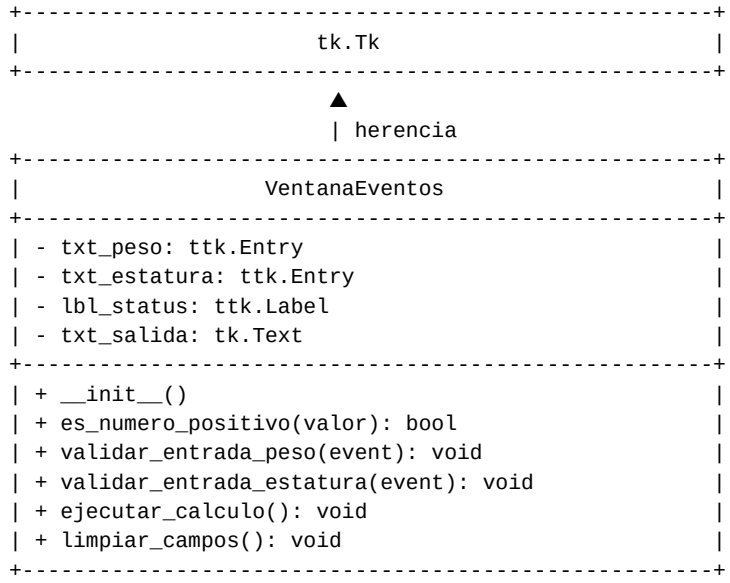
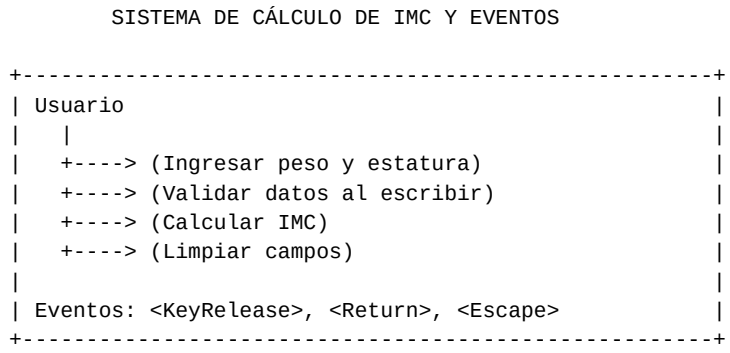


Diagrama de Casos de Uso UML



Descripción del funcionamiento

La clase VentanaEventos hereda de tk.Tk y contiene los campos de entrada, el mensaje de estado y el área de salida. Los métodos de validación comprueban que los datos puedan convertirse a números positivos. El método ejecutar_calculo calcula el IMC con la expresión peso / estatura², mientras que limpiar_campos reinicia la interfaz.

Referencias

Texto guía: *Ejercicios de Programación Orientada a Objetos con Java y UML*, de Leonardo Bermón Angarita.

Repositorio oficial en GitHub:

https://github.com/Kevin007ch/POO-Kevin-Santiago-Montoya-Chavarria

Nota: los dos programas se pueden guardar por separado como ejercicio_8_2.py y ejercicio_8_3.py y ejecutarse en un entorno de Python que tenga Tkinter disponible.